



文本处理工具

誉天教育

- 文本提取工具
- 文本分析工具
- 文本操作工具



- 文件内容：cat，more和less
- 文件摘选：head和tail
- 按关键字提取：grep
- 提取列或者字段：cut

- cat 打印一个或者多个文件内容

- A 显示所有的控制符

- n 显示排序号

- more 分页查看文件

- d 显示翻页和退出提示

- less 分页查看文件

- 查看时拥有的命令：

- /passwd 搜索 passwd

- n/N 跳到下一个或者上一个匹配

- less 是man命令使用的分页器

- head 列出指定文件前10行
 - n [k] 显示文件的前k行
- tail 列出指定文件的后10行
 - n [k] 显示文件的后k行
 - f 将文件追加的内容实时显示（排错常用，监控日志非常有用！）
- wc 文件数据统计
 - l 计算行数
 - w 计算单词数
 - c 计算字节数

- 查看/etc/passwd的第15到20行
 - `cat -n /etc/passwd|head -n20|tail -n6`
- 使用less查看/etc/passwd，并且过滤ftp
`less /etc/passwd`（进入后输入 / ftp）
- 使用cat显示/etc/passwd的行数
 - `cat -n /etc/passwd`

- grep文本过滤，通过关键字进行匹配
 - grep [options] 模式 [file...]
- 常用选项：
 - -v 反向过滤，打印不匹配的行
 - -I 忽略大小写
 - -c 统计匹配到的行数
 - -o 仅显示查找到的关键字
 - -q 静默模式，不输出任何信息
 - -Ax 将匹配行以及其后x行打印
 - -Bx 将匹配行以及其前x行打印
 - -Cx 将匹配行以及其前后x行打印
 - -r/R 根据匹配文本内容进行查找文件
 - -l 只打印文件名称，常与-rl使用，例如 grep -rl 'root' /etc/
 - --color=auto 高亮显示匹配到的关键字



- 正则表达式根据元字符的数量及功能不同又分为基本正则表达式（grep）和扩展正则表达式（egrep）。而grep和egrep同属于文本搜索工具，可根据用户指定的文本模式（搜索条件）对目标文件进行逐行搜索，显示能匹配到的行。用法上grep -E等同于egrep

字符	含义
^	在每行的开始进行匹配
\$	在每行的末尾进行匹配
\<	在字的开始进行匹配
\>	在字的末尾进行匹配
.	对任何单个字符进行匹配
[str]	对str中的任何单个字符进行匹配
[^str]	对任何不在str中的单个字符进行匹配
[a-b]	对a到b之间的任何字符进行匹配
\	抑止后面的一个字符的特殊含义
*	对前一项(item)进行0次或多次重复匹配

- 扩展正则表达式 (Extended Regular Expression , ERE) 支持比基本正则表达式更多的元字符，但是扩展正则表达式对有些基本正则表达式所支持的元字符并不支持。前面介绍的元字符 “^”、“\$”、“.”、“*”、“[]”以及 “[^]” 这6个元字符在 扩展正则表达式都得到了支持，并且其意义和用法都完全相同，不再重复介绍。接下来重点介绍一下在扩展正则表达式中新增加的一些元字符。
- egrep支持扩展表达式，相当于grep -E

字符	含义
+	对前一项进行1次或多次重复匹配
?	对前一项进行0次或1次重复匹配
{j}	对前一项进行j次重复匹配
{j,}	对前一项进行j次或更多次重复匹配
{,k}	对前一项最多进行k次重复匹配
{j,k}	对前一项进行j到k次重复匹配
s t	匹配s项或t项中的一项
(exp)	将exp作为单项处理

- 1) ^word 表示搜索以word开头的内容。
- 2) word\$ 表示搜索以word结尾的内容。
- 3) ^\$ 表示空行，不是空格。
- 4) . 代表且只能代表一个任意字符。
- 5) \ 转义字符，让有着特殊身份意义的字符。 例如：\.只表示小数点，还原原始的小数点的意义。
- 6) * 重复0个或多个前面的字符
- 7) .* 匹配所有的字符。 ^.* 任意多个字符开头。
- 8) [] 匹配字符集合内任意一个字符，如[a-z]
- 9) [^abc] ^在中括号里表示非，不包含a或b或c
- 10) {n,m} 匹配n到m次，前一个字符。 {n,} 至少N次，多了不限。 {n} N次 {m} 至多m次，少了不限。 注意：grep要{转义}，\{\},egrep不需要转义
- 11) \<或\b:锚定词首(支持vi和grep)，其后面的任意字符必须作为单词首部出现，如 \- 12) \>或\b:锚定词尾(支持vi和grep)，其前面的任意字符必须作为单词尾部出现，如 love\>或love\b

- cut 提取字段或列，从文件每一行剪切字节、字符、和字段并将这些字节、字符和字段输出
- 基本用法
 - cut [选项参数] filename
- 选项参数
 - f 列号 提取第几列
 - d 指定分隔符
 - c 按字符个数来提取
- 示例：
 - cut -c2-5 /etc/passwd #提取第2到第5个字符
 - cut -d: -f1 /etc/passwd #以冒号：为分隔符，提取第一列
 - ifconfig ens160 | grep netmask | cut -d " " -f 10 #提取ens160网卡IP地址



文本分析工具

- 文本统计: `wc`
- 文本排序: `sort`
- 文件比较: `diff`



Red Hat



誉天教育
YU TIAN EDU





wc - 单词统计

- 计算单词数，行数，字节数和字符数
- 可以针对一个文件或者标准输入

```
$ wc story.txt
```

```
39 237 1901 story.txt
```

- 常用选项：
 - l：仅仅统计行数
 - w：仅仅统计单词数
 - c：仅仅统计字节数



Red Hat



誉天教育
YU TIAN EDU

➤ sort 文本排序，对标准输出进行排序，原文件不改动

➤ 基本用法

- sort [选项参数] filename

➤ 选项参数

-r 倒序

-n 按照数字大小进行排序

-u 删除重复行

-f 忽略大小写

-t c 使用c作为字段间的分隔符

-k x 使用c分隔符排序第x字段

sort和uniq - 消除重复行

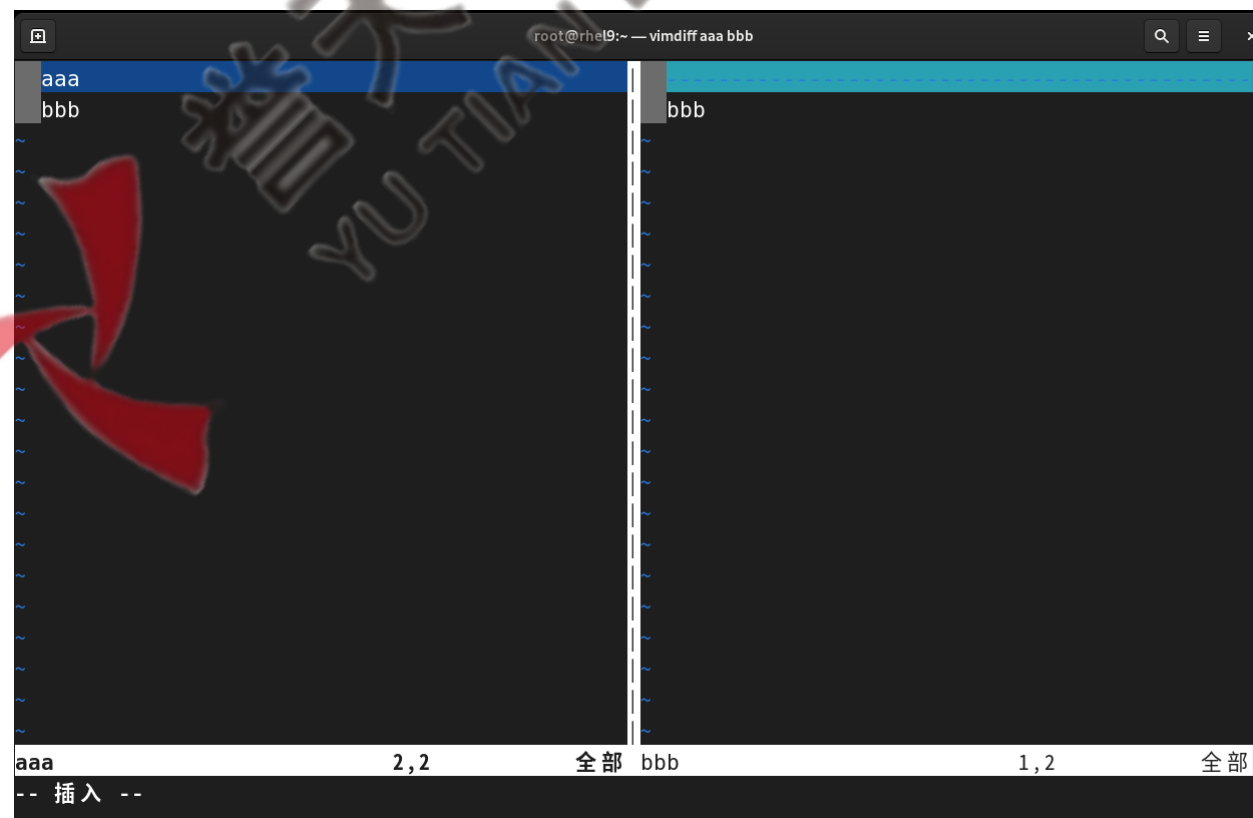
- sort -u : 从输入删除重复行
- uniq : 从相邻的行中删除重复行
 - ◆ 使用-c选项统计发生重复的次数
 - ◆ 跟sort一起使用效果最好 : `$ sort userlist.txt | uniq -c`

- diff 比较文件不同
- 基本用法
 - diff filename1 filename2
- 示例：aaa文件里内容为aaa bbb，bbb文件里内容为bbb

```
[root@rhel9 ~]# cat aaa
aaa
bbb
[root@rhel9 ~]# cat bbb
bbb
[root@rhel9 ~]# diff aaa bbb
1d0
< aaa
```


文本比较工具-vimdiff

- vimdiff 图形化比较文件不同 (来源自vim-enhanced)
- 基本用法
 - vimdiff filename1 filename2
- 示例：aaa文件里内容为aaa bbb，bbb文件里内容为bbb



➤ tr 用于字符转换、删除

- 转换一种字符集合为另外一种字符集合
- 只能从STDIN读取数据

- `$ tr 'a-z' 'A-Z' < /etc/passwd`

➤ 示例：

- `$ tr 'a-z' 'A-Z' < /etc/passwd` #把小写字母转换为大写字母

- `$ tr -d 'root' < /etc/passwd` #删除/etc/passwd中的root字符串

- sed 是一种用于文本处理的强大工具。它可以执行文本替换、删除、插入和转换等操作
- 一次处理一行
- 处理数据时先放入缓冲区，也是模式空间，经过sed处理后再输出到屏幕
- 不改变源文件，通过 -i.bak备份和修改源文件



➤ 语法格式：

- sed OPTIONS... [SCRIPT] [INPUTFILE...]

➤ 常用选项：

- n , --quiet 不打印模式空间内容
- i 直接编辑文件内容，默认不对原文件改变
- e 添加（多个）脚本进行操作
- r 使用扩展正则表达式

- 1) # : #为数字, 指定要进行处理操作的行
- 2) \$: 表示最后一行, 多个文件进行操作的时候, 为最后一个文件的最后一行
- 3) /regexp/ : 表示能够被regexp匹配到的行, regexp及基于正则表达式的匹配
- 4) /regexp/I : 匹配时忽略大小写
- 5) \%regexp%: 任何能够被regexp匹配到的行, 换用% (用其他字符也可以, 如: #) 为边界符号
- 6) addr1,addr2 : 指定范围内的所有的行 (范围选定) 常用地址定界表示方式
 - a) 0, /regexp/ : 从起始行开始到第一次能够被regexp匹配到的行
 - b) /regexp/,/regexp/ : 被模式匹配到的行内的所有的行
- 7) first~step : 指定起始的位置及步长, 例如: 1~2表示1,3,5...
- 8) addr1,+N : 指定行以及以后的N行
 - addr1,~N : 指定行开始的N行

- 常用编辑命令

- 1) p : 打印模式空间中的内容

- 2) d : 删除匹配到的行

- 3) a \text : append,表示在匹配到的行之后追加内容

- 4) i \text : insert,表示在匹配到的行之前追加内容

- 5) c \text : change,表示把匹配到的行和给定的文本进行交换

- 6) s/regexp/replacement/flags : 查找替换,把text替换为 regexp匹配到的内容 (其中/可以用其他字符代替, 例如@)

- 其他编辑命令 :

- g : 全局替换, 默认只替换第一个

- i : 不区分大小写

- p : 如果成功替换则打印

- 7) w /path/to/somefile : 将匹配到的文件另存到指定的文件中

- 注意事项:

- 1、如果没有指定地址，表示命令将应用于每一行
- 2、如果只有一个地址，表示命令将应用于这个地址匹配的所有行
- 3、如果指定了由逗号分隔的两个地址，表示命令应用于匹配第一个地址和第二地址之间的行(包括这两行)
- 4、如果地址后面跟有感叹号，表示命令将应用于不匹配该地址的所有行

案例1

- 示例

`sed -n 5p passwd` 打印第5行

`sed -n '$p' passwd` 打印最后一行

`sed -n '1,5p' passwd` 打印第1-5行

`sed -n '/root/Ip' passwd` 匹配带有root关键词的行，并忽略大小写

`sed -n '%root%Ip' passwd` 同上

`sed '1,5d' passwd` 删除第1-5行

`sed -i.bak '1,5d' passwd` 删除第1-5行，源文件被修改

`sed '2a \abc' passwd` 在文件第2行下面追加abc

示例

sed 's/north/hello/' datafile --替换每行第一个north

sed 's/north/hello/g' datafile --全部替换

sed '1 s/north/hello/g' datafile --替换第一行所有的north

sed '1 s/north/hello/' datafile --替换第一行第一个north

sed '1 s/north/hello/2' datafile --只替换第一行第二个north

巧用替换删除内容（不是删除行）

sed 's/north//' datafile --删除所有行的第一个north

sed 's/north//g' datafile --删除全部north

sed '1 s/north//2' datafile --删除第一行第二个

sed 's/^/#/' datafile 给每行开始加注释，^就代表开始

sed 's/^./' datafile --删除每行第一个字母

sed 's/^\(..\).^1/' datafile --删除第3个字母 注释：\1代表第一个括号匹配到的内容

sed 's/^\<[a-Z]*[a-Z]\>/' datafile --删除每行第一个单词

- ✓ 文本查看工具
- ✓ 文本过滤工具
- ✓ 文本处理工具

